

Taller 7

Parte 1: Despliegue de la API con contenedores en EC2

1.1 Instancia en AWS EC2

The screenshot shows the AWS Management Console interface for EC2 instances. The left sidebar contains navigation links for EC2, Instancias, and Imágenes. The main content area displays a table of instances. One instance, 'Servidor_Taller7', is listed with ID 'i-0bae28da1dce7fa0e', state 'En ejecución', and type 't3.micro'. Below the table, the details for this instance are shown, including its public IP address (54.165.36.15) and private IP address (172.31.16.97).

Nombre	ID de la instancia	Estado de la instancia	Tipo de instancia	Comprobación de estado	Zona de disponibilidad	DNS de la instancia
Servidor_Taller7	i-0bae28da1dce7fa0e	En ejecución	t3.micro	3/3 checks passec	us-east-1b	ec2-54-165-36-15.us-east-1b.amazonaws.com

i-0bae28da1dce7fa0e (Servidor_Taller7)

Resumen de instancia

- ID de la instancia: i-0bae28da1dce7fa0e
- Dirección IPv4 pública: 54.165.36.15 | dirección abierta
- Direcciones IPv4 privadas: 172.31.16.97
- Dirección IPv6: -
- Estado de la instancia: En ejecución
- DNS público: ec2-54-165-36-15.us-east-1b.amazonaws.com

1.11 Pantallazo y explicación de salida de Docker hello-world

```
ubuntu@ip-172-31-16-97:~$ sudo docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f55086f7dd0: Pull complete
d5e71e642bf5: Download complete
Digest: sha256:c3cbe1cc1aa588a64951ac6286e0df7b27fe2e6324b1001c619bb358770c0178
Status: Downloaded newer image for hello-world:latest
```

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:

1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub. (amd64)
3. The Docker daemon created a new container from that image which runs the executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
\$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
<https://hub.docker.com/>

For more examples and ideas, visit:
<https://docs.docker.com/get-started/>

```
ubuntu@ip-172-31-16-97:~$
```

El mensaje de salida valida que el servicio dockerd tiene permisos suficientes para gestionar imágenes y ejecutar contenedores en el sistema operativo Ubuntu de la instancia AWS EC2.

1.13 Explicación de Dockerfile

El Dockerfile es el manual de instrucciones para construir la imagen del contenedor

- **FROM python:3.12-slim:** Define la imagen base sobre la cual se construye el contenedor. Se utiliza la versión slim de Python 3.12
- **RUN adduser ...:** Ejecuta un comando para crear un usuario llamado api-user.
- **RUN apt-get update && apt-get install ...:** Instala herramientas de compilación (build-essential) necesarias en el sistema operativo.
- **WORKDIR /opt/bankchurn-api:** Establece el directorio de trabajo dentro del contenedor donde se realizarán todas las operaciones posteriores.
- **ADD ./bankchurn-api /opt/bankchurn-api/:** Copia los archivos del código fuente desde la máquina hacia el directorio de trabajo dentro del contenedor.
- **RUN pip install --upgrade pip y RUN pip install -r ...:** Actualiza el gestor de paquetes de Python e instala todas las dependencias definidas en tu archivo requirements.txt.
- **RUN chmod +x ...:** Modifica los permisos del archivo run.sh para asegurar que el contenedor tenga permisos de ejecución sobre el script que inicia la aplicación.
- **RUN chown -R api-user:api-user ./:** Cambia la propiedad de los archivos y carpetas del proyecto al usuario api-user creado anteriormente, asegurando que solo él pueda manipularlos.
- **USER api-user:** Cambia el usuario activo del contenedor a api-user para todas las instrucciones que siguen, garantizando que la aplicación corra bajo este usuario restringido.
- **EXPOSE 8001:** Documenta que el contenedor está configurado para escuchar conexiones en el puerto 8001.
- **CMD ["bash", "./run.sh"]:** Es el comando final que se ejecuta automáticamente cuando lanzas un contenedor a partir de esta imagen. En este caso, ejecuta el script run.sh, el cual contiene la instrucción de lanzamiento de la aplicación con FastAPI.

1.16 Pantallazo de salida de docker images

```
ubuntu@ip-172-31-16-97:~/Taller7-Docker$ sudo docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
bankchurn-api:latest	44d77d63aa6	1.46GB	396MB	
hello-world:latest	c3cbelcc1aa5	25.9kB	9.49kB	U

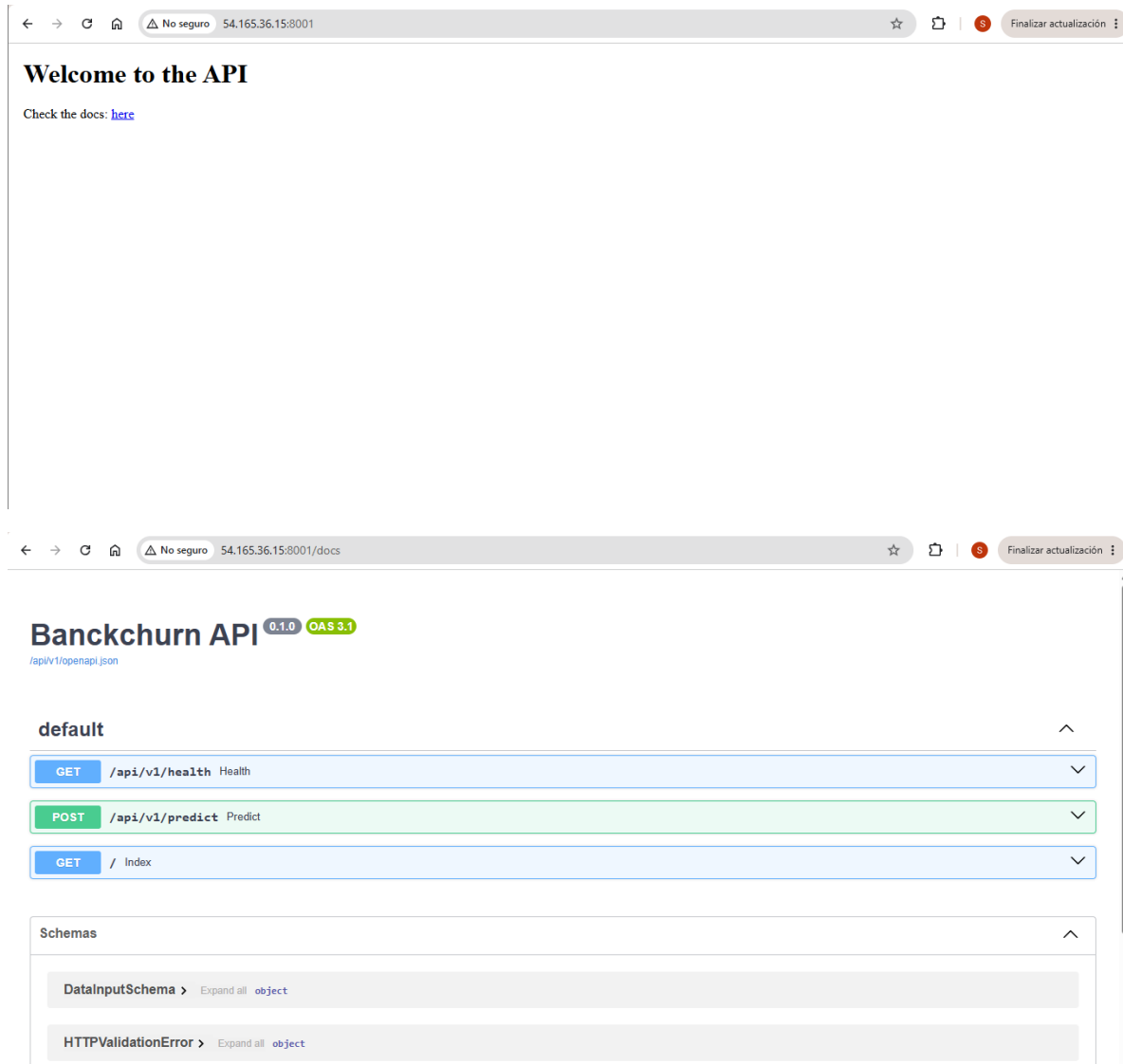
```
ubuntu@ip-172-31-16-97:~/Taller7-Docker$ |
```

1.17 Pantallazo de salida de docker run

```
ubuntu@ip-172-31-16-97:~/Taller7-Docker$ sudo docker run -p 8001:8001 -e PORT=8001 bankchurn-api
```

```
INFO:      Started server process [7]
INFO:      Waiting for application startup.
INFO:      Application startup complete.
INFO:      Uvicorn running on http://0.0.0.0:8001 (Press CTRL+C to quit)
```

1.20 Pantallazo de navegador con API en ejecución



Parte 2: Despliegue del tablero con contenedores en EC2

2.6 Pantallazo de salida de docker images

```
ubuntu@ip-172-31-16-97:~/Taller7-Docker/docker-dash-starter$ sudo docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
bankchurn-api:latest	ed126c397cb3	1.46GB	396MB	U
bankchurn-dash:latest	d3daa354d108	1.46GB	396MB	U
hello-world:latest	c3cbe1cc1aa5	25.9kB	9.49kB	U

```
ubuntu@ip-172-31-16-97:~/Taller7-Docker/docker-dash-starter$
```

2.7 Pantallazo de salida de docker run

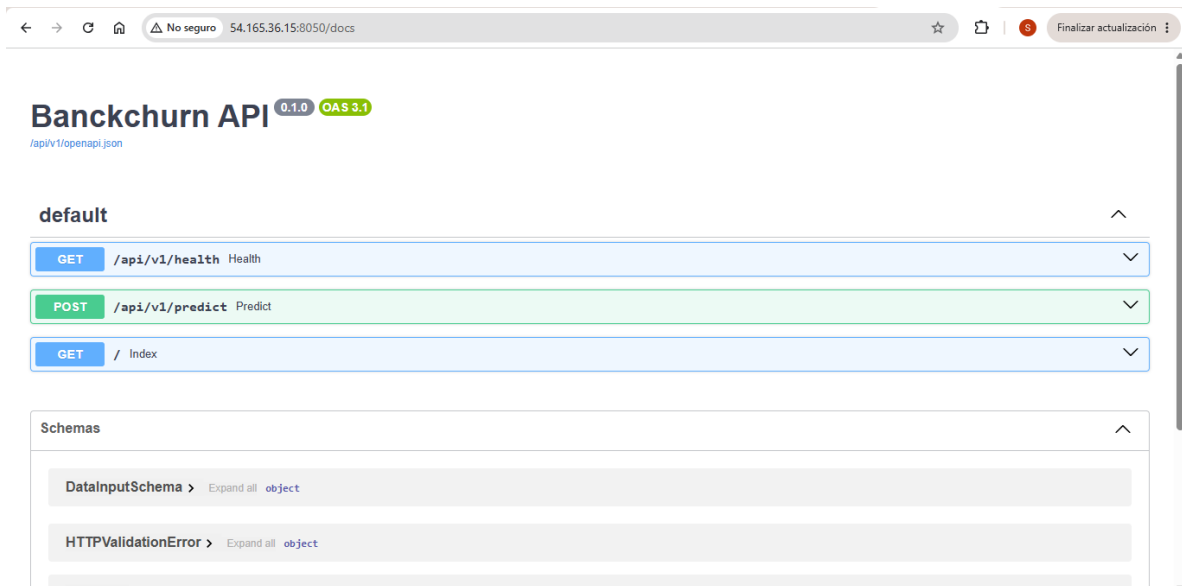
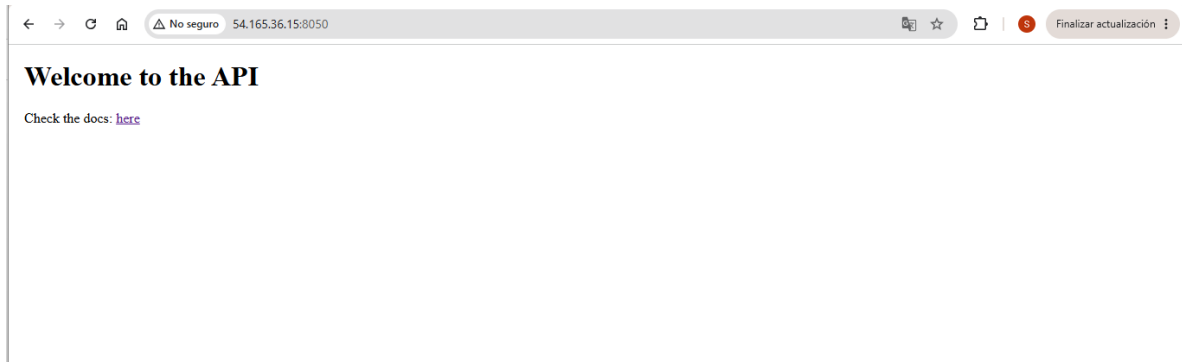
```
ubuntu@ip-172-31-16-97:~/Taller7-Docker/docker-dash-starter$ sudo docker run -p 8050:8050 -e PORT=8050 -e API_URL=http://TU_IP_PUBLICA:8001 -e API_P
ORT=8001 bankchurn-dash
INFO: Started server process [8]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:8050 (Press CTRL+C to quit)
```

```

ubuntu@ip-172-31-16-97:~/Taller7-Docker/docker-dash-starter$ sudo docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
4c61dc3f76ec   bankchurn-dash "bash ./run.sh"         2 minutes ago Up 2 minutes   8001/tcp, 0.0.0.0:8050->8050/tcp, [::]:8050->8050/tcp   youthful_wilson
c15bc8365086   444d77d63aae6 "bash ./run.sh"         34 minutes ago Up 34 minutes   0.0.0.0:8001->8001/tcp, [::]:8001->8001/tcp             funny_turing
ubuntu@ip-172-31-16-97:~/Taller7-Docker/docker-dash-starter$

```

2.10 Pantallazo de tablero y terminales de los dos contenedores en ejecución



```

ubuntu@ip-172-31-16-97:~/Taller7-Docker/docker-dash-starter$ sudo docker logs funny_turing --tail 20
INFO: Started server process [7]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:8001 (Press CTRL+C to quit)
2026-07-16 04:42:13.447 | INFO | logging:callHandlers:1762 - 191.104.159.252:51735 - "GET / HTTP/1.1" 200
2026-07-16 04:42:13.782 | INFO | logging:callHandlers:1762 - 191.104.159.252:51735 - "GET /favicon.ico HTTP/1.1" 404
2026-07-16 05:17:07.861 | INFO | logging:callHandlers:1762 - 191.104.159.252:57965 - "GET /docs HTTP/1.1" 200
2026-07-16 05:17:08.404 | INFO | logging:callHandlers:1762 - 191.104.159.252:57965 - "GET /api/v1/openapi.json HTTP/1.1" 200
ubuntu@ip-172-31-16-97:~/Taller7-Docker/docker-dash-starter$ sudo docker logs youthful_wilson --tail 20
INFO: Started server process [8]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:8050 (Press CTRL+C to quit)
2026-07-16 05:16:59.985 | INFO | logging:callHandlers:1762 - 191.104.159.252:57205 - "GET / HTTP/1.1" 200
2026-07-16 05:17:00.238 | INFO | logging:callHandlers:1762 - 191.104.159.252:57205 - "GET /favicon.ico HTTP/1.1" 404
2026-07-16 05:17:02.985 | INFO | logging:callHandlers:1762 - 191.104.159.252:57205 - "GET /docs HTTP/1.1" 200
2026-07-16 05:17:03.816 | INFO | logging:callHandlers:1762 - 191.104.159.252:57205 - "GET /api/v1/openapi.json HTTP/1.1" 200
2026-07-16 05:17:52.400 | INFO | logging:callHandlers:1762 - 191.104.159.252:49729 - "GET / HTTP/1.1" 200
2026-07-16 05:18:06.853 | INFO | logging:callHandlers:1762 - 191.104.159.252:60609 - "GET /docs HTTP/1.1" 200
2026-07-16 05:18:07.137 | INFO | logging:callHandlers:1762 - 191.104.159.252:60609 - "GET /api/v1/openapi.json HTTP/1.1" 200
ubuntu@ip-172-31-16-97:~/Taller7-Docker/docker-dash-starter$

```

2.11 Comparación archivos Dockerfile

Característica	bankchurn-api	bankchurn-dash
Directorio de trabajo	/opt/bankchurn-api	/opt/app
Usuario ejecutor	api-user	dash-user
Puerto expuesto	8001	8050
Fuente de archivos	./bankchurn-api	./app

Tres diferencias:

- **Exposición de Puertos:** La API expone el puerto 8001 y el Tablero el 8050. Esto se requiere porque ambos servicios necesitan estar accesibles simultáneamente desde la máquina virtual y el exterior, por lo que deben utilizar canales de comunicación distintos para evitar conflictos de *binding* en el sistema operativo.
- **Directorio y Usuario:** Los Dockerfiles están configurados para operar en entornos aislados. Mientras que la API utiliza el directorio /opt/bankchurn-api y el usuario api-user, el tablero opera bajo /opt/app y dash-user. Esta práctica sigue el principio de menor privilegio, asegurando que si un contenedor fuera comprometido, el daño quede confinado a sus archivos específicos sin afectar al otro servicio.
- **Gestión de Dependencias y Código:** El contenedor de la API importa su código desde la carpeta local ./bankchurn-api, mientras que el tablero lo hace desde ./app. Esto demuestra una arquitectura modular donde cada contenedor es responsable de gestionar sus propias dependencias (requirements.txt) y scripts de ejecución (run.sh), permitiendo que el tablero consuma la API como un servicio externo